Lab 2: Problem Solving using Informed Search: A* Search for 8 Puzzle

Problem Statement

The **8-Puzzle Problem** consists of a 3×3 board containing eight numbered tiles and one blank space (0). The objective is to move the tiles by sliding them into the blank space until the goal configuration is reached.

Use the **A*** Search Algorithm to solve the 8-puzzle problem using two heuristics:

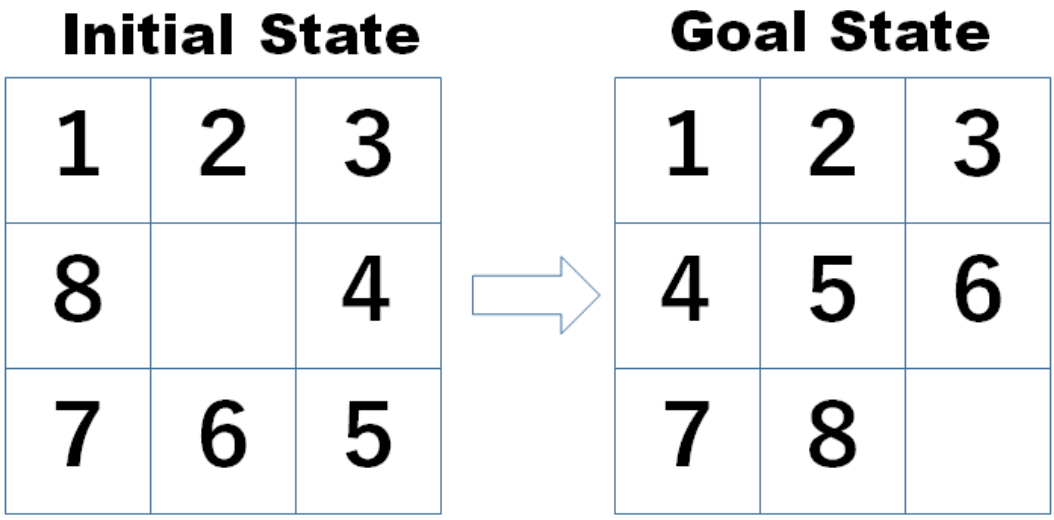
- **h_1(n): Number of Misplaced Tiles**
- **h_2(n): Manhattan Distance**

Compare the performance of both heuristics.

Problem Formulation

State Space

All possible arrangements of eight numbered tiles and one blank space.



Successor Function

Generate all valid states by moving the blank tile:

- Up
- Down
- Left
- Right

Path Cost

Each move has a cost of **1**.

Goal Test

Check whether the current state matches the goal state.

Evaluation Function

$$\mathbf{f(n)=g(n)+h(n)}$$

where:

- g(n) = cost from start state to current state
- h(n) = estimated cost from current state to goal state

Heuristic 1: Misplaced Tiles (h_1)
Count the number of tiles that are not in their goal position.

$$h_1(n) = \text{Number of misplaced tiles}$$

Heuristic 2: Manhattan Distance (h_2)

Sum of horizontal and vertical distances of each tile from its goal position.

$$h_2(n) = \sum |x_i - x_g| + |y_i - y_g|$$

Source Code

```
from heapq import heappush, heappop


goal = (
(1, 2, 3),
(4, 5, 6),
(7, 8, 0)
)

start = (
(5, 8, 2),
(1, 0, 3),
(4, 7, 6)
)

def misplaced_tiles(state):
	count = 0
	for i in range(3):
		for j in range(3):
			if state[i][j] != 0 and state[i][j] != goal[i][j]:
				count = count + 1
	return count

def find_blank(state):
	for i in range(3):
		for j in range(3):
			if state[i][j] == 0:
				return i,j

def get_neighbors(state):
	x, y = find_blank(state)
	moves = [(-1,0), (1,0), (0,-1), (0,1)]
	neighbors = []
	for dx, dy in moves:
		nx = x + dx
		ny = y + dy
		if 0 <= nx < 3 and 0 <= ny < 3:
			board = [list(row) for row in state]
			board[x][y], board[nx][ny] = board[nx][ny], board[x][y]
			neighbors.append(tuple(map(tuple, board)))
	return neighbors
```

```
def reconstruct(parent, current):
	path = []
	while current is not None:
		path.append(current)
		current = parent[current]
	return path[::-1]

def astar(start, heuristic):
	pq = []
	heappush(pq, (heuristic(start), 0, start))
	parent = {start: None}
	g_cost = {start: 0}
	expanded = 0
	while pq:
		f, g, current = heappop(pq)
		expanded = expanded + 1
		if current == goal:
			return reconstruct(parent, current), expanded
		for neighbor in get_neighbors(current):
			new_g = g + 1
			if neighbor not in g_cost or new_g < g_cost[neighbor]:
				g_cost[neighbor] = new_g
				f_cost = new_g + heuristic(neighbor)
				heappush(
				pq,
				(f_cost, new_g, neighbor)
				)
				parent[neighbor] = current
	return None, expanded

print("Using h1: Misplaced Tiles")
path, expanded = astar(start, misplaced_tiles)
if(path):
	for step in path:
		for row in step:
			print(row)
		print()

	print("Moves = ",len(path)-1)

print("Nodes Expanded = ", expanded)
```

```
Using h1: Misplaced Tiles
(5, 8, 2)
(1, 0, 3)
(4, 7, 6)

(5, 0, 2)
(1, 8, 3)
(4, 7, 6)

(0, 5, 2)
(1, 8, 3)
(4, 7, 6)

(1, 5, 2)
(0, 8, 3)
(4, 7, 6)

(1, 5, 2)
(4, 8, 3)
(0, 7, 6)

(1, 5, 2)
(4, 8, 3)
(7, 0, 6)

(1, 5, 2)
(4, 0, 3)
(7, 8, 6)

(1, 0, 2)
(4, 5, 3)
(7, 8, 6)

(1, 2, 0)
(4, 5, 3)
(7, 8, 6)

(1, 2, 3)
(4, 5, 0)
(7, 8, 6)

(1, 2, 3)
(4, 5, 6)
(7, 8, 0)

Moves =  10
Nodes Expanded =  30
```

Task 1: Implement Manhattan Distance Heuristic (h2)
Task 2: Implement Combined Heuristic (h1 + h2)
Task 3: Compare the Heuristics by runnig the A* algorithm using:

1. h1: Misplaced Tiles
2. h2: Manhattan Distance
3. h1 + h2: Combined Heuristic